

The Snowflake Elastic Data Warehouse

Архитектура cloud-native аналитической СУБД, SIGMOD 2016

Service-Oriented Architecture

Separation of Concerns

Immutability & MVCC

Проблема: shared-nothing архитектура не подходит для облака

Гетерогенность нагрузки

Конфигурация для bulk load (высокий I/O) — плохой fit для аналитических запросов (CPU). Результат: компромиссное железо с низкой средней утилизацией.

Membership changes

Добавление / удаление узла требует перераспределения данных. Те же узлы, что делают reshuffling, обрабатывают запросы — система деградирует.

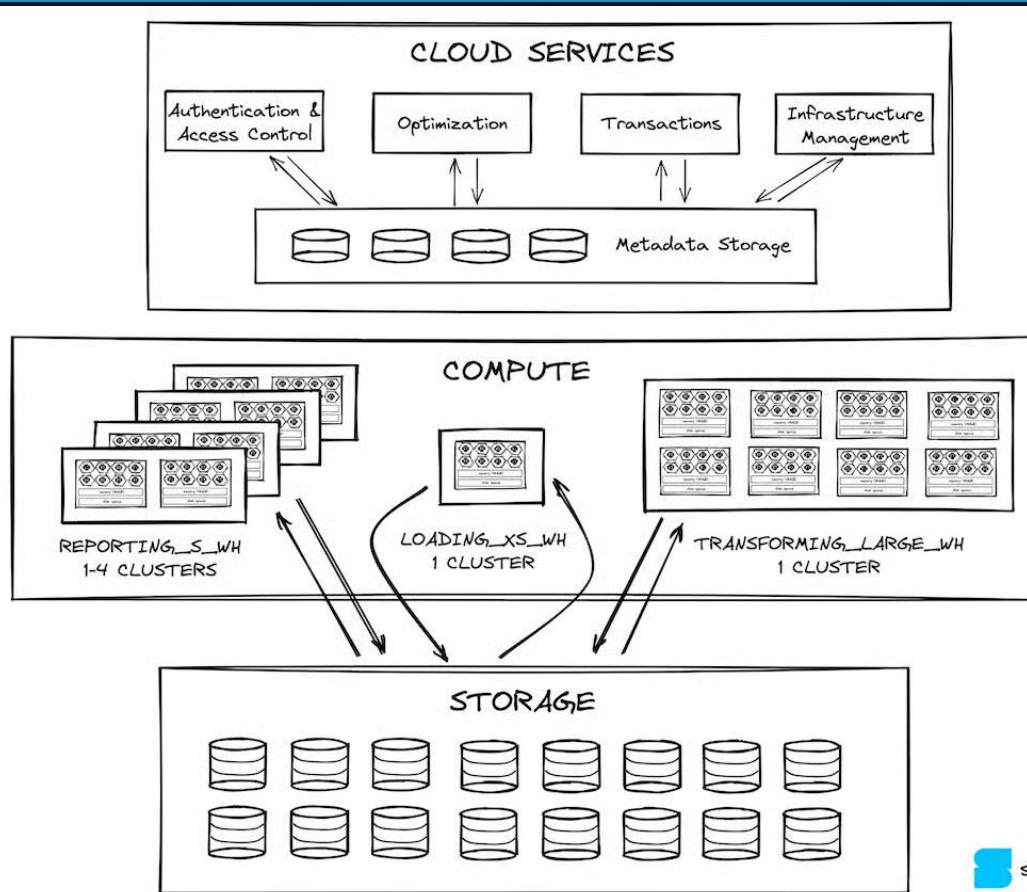
Online upgrade невозможен

Всё тесно связано и предполагается гомогенным. Обновление системы без даунтайма практически нереализуемо.

Корень проблемы:

Compute и Storage жёстко связаны. В on-premise это терпимо — нагрузка предсказуема, узлы редко меняются. В облаке — membership changes норма, нагрузка гетерогенна, есть сильный стимул к online upgrade. Вывод авторов: нужна фундаментальная перепроектировка.

Решение: Service-Oriented Architecture с тремя НЕЗАВИСИМЫМИ СЛОЯМИ



Storage Layer: PAX-формат, иммутабельность и min-max pruning

S3 диктует дизайн данных

S3 — blob store: только PUT/GET/DELETE, нет append, нет partial write. Файл нельзя изменить — только перезаписать целиком. Это ограничение напрямую определило формат хранения.

PAX (hybrid columnar)

Таблицы → иммутабельные файлы. Внутри файла: значения каждой колонки сгруппированы (PAX). Range GET из S3 → только нужные колонки. Тяжёлое сжатие по колонкам.

MVCC из иммутабельности

UPDATE = новый файл + пометка старого. Нет concurrent write conflicts, нет undo log, нет buffer pool. Snapshot Isolation тривиален: транзакция видит snapshot на момент старта.

Min-Max Pruning (Zone Maps)

Файл	min(date)	max(date)	min(amount)	max(amount)
f1	2024-01	2024-03	10	500
f2	2024-03	2024-06	5	900
f3	2024-07	2024-12	20	300

Запрос: WHERE date > '2024-06'

f1 (max=2024-03 < 2024-06) → SKIP

f2 (max=2024-06 = граница) → SCAN

f3 (min=2024-07 > 2024-06) → SCAN

Также: dynamic pruning — bloom filters по join keys во время выполнения запроса

Virtual Warehouses: изоляция, consistent hashing и file stealing

**VW: BI
(XS)**

Worker nodes не шарятся

**VW: ETL
(L)**

Worker nodes не шарятся

**VW: DS
(XXL)**

Worker nodes не шарятся

☁ Единое хранилище — Amazon S3 (все VW читают одни и те же файлы)

Consistent hashing + LRU

Один файл → всегда к одной ноде (по хешу имени) → высокий cache hit. При изменении состава нод — lazy: кэш вытесняется постепенно. Доступность > немедленная оптимальность.

Ephemeral worker processes

Каждый запрос → отдельный worker process, живёт только пока запрос. Иммуемые файлы → нет visible side effects → отказ ноды = перезапуск запроса. Без recovery состояния.

File stealing (skew handling)

Нода завершила файлы → запрашивает их у «медленных» нод. Читает прямо из S3, не нагружая straggler. Решение проблемы виртуализационных задержек в облаке.

Cloud Services: Cascades-оптимизатор и online upgrade без даунтайма

Query Optimizer (Cascades)

Top-down cost-based

Cascades framework — оптимизация сверху вниз по дереву плана с оценкой стоимости

Нет индексов — осознанно

Меньше пространство поиска планов → производительность предсказуема. Snowflake: "только один tuning parameter — сколько compute"

Отложенные решения

Тип join distribution решается в runtime, не в optimizer → меньше плохих планов за счёт небольшой потери пиковой производительности

Статистика автоматически

Обновляется при загрузке данных. Нет ANALYZE, нет VACUUM — zero knobs для пользователя

Online Upgrade без даунтайма

1

Deploy v2 рядом с v1 (side-by-side). Все сервисы stateless — состояние только в KV-store.

2

Load balancer постепенно переключает аккаунты на v2. Новые запросы → v2.

3

Старые запросы доживают на v1. VW разных версий могут шарить worker nodes и кэш.

4

Все перешли на v2 → v1. Rollback = та же процедура в обратную сторону.

Результат: деплой каждую неделю, 0 даунтайма (на момент публикации статьи)

ELT вместо ETL и columnar / vectorized / push-based execution engine

ELT-паттерн и тип VARIANT

ETL (традиционно)

Transform → Load. Смена схемы = координация между отделами, месяцы задержки.

ELT (Snowflake)

Load → Transform позже. VARIANT: JSON/Avro/XML без schema. Schema inference автоматически.

Decoupling producers/consumers

Producer льёт новые поля — consumer читает когда готов. Архитектурный принцип разделения ответственностей.

Columnar хранение JSON

Snowflake статистически анализирует документы в каждом файле, выносит частые поля в отдельные колонки. Overhead schema-less vs relational — ~10% по TPC-H бенчмарку.

Execution Engine

COLUMNAR

Данные и выполнение по колонкам. Эффективнее CPU caches, SIMD-инструкции, сжатие. Класс систем: C-store, MonetDB.

VECTORIZED

Обработка батчами ~тысяч строк без материализации промежуточных результатов (в отличие от MapReduce). Идея из MonetDB/X100.

PUSH-BASED

Операторы толкают данные вниз по плану (vs Volcano pull). Убирает control flow из tight loops → лучше cache efficiency. Поддерживает DAG-планы, а не только деревья.

NO BUFFER POOL

Нет традиционного buffer pool — запросы сканируют большие данные, буферизация таблиц невыгодна. Spill-to-disk через S3 для очень больших join / sort.

Выводы: архитектурные уроки из статьи

01

Ограничения платформы диктуют дизайн

S3 не поддерживает append → иммутабельные файлы → MVCC тривиален → нет buffer pool → нет coordinator write conflicts. Одно техническое ограничение развернуло весь дизайн данных. Проектирование начинается с constraints.

02

Separation of Concerns — физически, не логически

Три слоя с разными lifecycle: storage (permanent) / compute (ephemeral) / services (long-lived). Разные команды, разные деплои. Следствие: online upgrade каждую неделю без даунтайма. SRP applied to infrastructure.

03

Осознанные trade-offs, а не «лучшие практики»

Lazy consistent hashing: доступность > немедленная оптимальность кэша. Отказ от индексов: предсказуемость > пиковая производительность. Отложенные решения в оптимизаторе: меньше плохих планов > максимальная оптимизация.

04

Влияние: одна идея переформатировала индустрию

Shared storage + independent compute → BigQuery, Redshift Serverless, Azure Synapse. ELT-паттерн вместо ETL → dbt, Databricks. Принципы иммутабельности и stateless services → современная cloud-native архитектура.